

Homeowner Loss History Prediction

EECS 4020 Senior Design I & II Final Report

Team Members:

Ahmad Firas Azfar, Ahmad Fer Rouse

Pranshu Chawla

Yashraj Mote

Faculty Advisors:

Dr. Zhiwei Chen

Dr. Syed Fayeeg

Course Instructor:

Dr. Liang Cheng

Industry Sponsor (Grange Insurance):

Greg Helser

April 28, 2025

Project Highlights

- Full-stack MLOps automation for loss prediction deployed on AWS
- Self-healing pipeline with schema validation and drift detection
- Manual override and alerting interface built for actuaries and analysts
- Over 100 features validated and versioned with reproducible snapshots
- Human-in-the-loop decision point tied into the training workflow
- Integration with Slack, MLflow, Prometheus, and Airflow for observability

End-to-End System Automation Snapshot

- **Ingestion:** Raw homeowner loss data → schema validation → preprocessing
- **Automation:** Airflow DAGs control all stages; MLflow tracks results
- **Monitoring:** Prometheus + Slack for real-time alerting and observability
- **Decision Layer:** Human-in-the-loop controls and manual override dashboard
- **Recovery:** Self-healing triggered on detected drift
- **Scalability:** Containerized, cloud-native deployment (EC2 + S3 + Lambda)

Abstract

This project presents a comprehensive MLOps automation system for predicting homeowner loss history, developed in collaboration with Grange Insurance. The system is designed to streamline and automate every phase of the machine learning lifecycle, including data ingestion, validation, preprocessing, drift detection, model training, evaluation, versioning, and deployment. Built on a modular AWS-based architecture, it utilizes Airflow for orchestration, MLflow for model tracking and registry, Prometheus, and Slack integration for real-time observability and alerting.

Our pipeline incorporates built-in schema validation (using Pandera), scalable preprocessing logic, monotonic hyperparameter tuning (via Hyperopt), and automated retraining triggered by detected data drift. A dedicated dashboard allows for manual intervention through a human-in-the-loop workflow, ensuring transparency and override capabilities at critical decision points. All components are containerized and adhere to CI/CD principles for maintainability and reproducibility.

Key outcomes include a 75% reduction in iteration time, a threefold improvement in retraining responsiveness, and the integration of over 100 engineered features with schema snapshotting and version control. The system enhances risk mitigation through improved model accuracy, automated drift response, and scalable deployment strategies—transforming a previously manual, high-friction modeling process into a governed, audit-ready, production-grade AI platform.

Contents

Glossary / Acronyms	8
1 Introduction	9
1.1 Motivation & Industry Need	9
1.2 Objective Statement & KPIs	9
1.3 Guiding Principles	10
1.4 Project Scope: Senior Design I vs II	11
1.5 Stakeholders & User Personas	11
1.6 Objective Tree Diagram	12
1.7 House of Quality Matrix	13
1.8 Problem Decomposition Diagram	13
2 Background & Prior Work	14
2.1 Industry Context and Motivation	14
2.2 Literature Review: Loss Prediction Models	14
2.3 Industry Practices in Rating	15
2.4 Tooling & Frameworks for MLOps	15
2.5 Regulatory & Ethical Considerations	16
2.6 Integrated View of Rating and MLOps	16
3 Alternative Designs & Risk Tradeoffs	17
3.1 Explored Designs	17
3.2 Concept Evaluation Matrix	18
3.3 Decision Matrix for Final Model Selection	19
3.4 Final Design Justification	19
3.5 Risk Mitigation Strategy	20
3.6 Comparative Design Table	21
4 System Architecture	21
4.1 Level 0 Design (Big Picture)	22
4.2 Level 1 Logical Design (Expanded Module Breakdown)	24
4.3 Level 1 Physical Deployment (AWS Infrastructure)	25
4.4 Human-in-the-Loop Automation	26
5 Software Implementation	26
5.1 Codebase Structure Overview	26
5.2 Airflow DAG Orchestration	27
5.3 Training Module: XGBoost + Hyperopt Integration	29
5.4 Drift Detection and Self-Healing Workflow	29

5.5	Slack-Based Human Oversight and Agent Actions	29
5.6	Data Profiling and Monitoring	29
6	UI & Dashboard	30
6.1	Goals & Design Philosophy	30
6.2	Mockups & Screenshots	31
6.3	Interactive UX Table	32
6.4	Manual Override Flow	32
6.5	User Feedback Summary	33
7	Testing & Validation	34
7.1	Unit Testing	34
7.2	Integration Testing	35
7.3	Performance Metrics	35
7.4	Drift Simulation Results	36
7.5	User Acceptance Testing (UAT)	36
7.6	Validation Table	37
8	Results & Discussion	37
8.1	Model Metrics Summary	37
8.2	Feature Importance Insights	38
8.3	Benefits Observed	39
8.4	Limitations	40
8.5	Lessons Learned	40
9	Business, Cost, & Commercial Analysis	41
9.1	Timeline & Work Breakdown	41
9.2	Final BOM & Cost Report	42
9.3	Economic Feasibility & ROI	42
9.4	Commercial Use Potential	43
9.5	Marketing & Sales Implications	44
9.6	Business Value Comparison Table	45
10	Regulatory, Ethical & Social Considerations	45
10.1	Data Privacy & Model Explainability	45
10.2	Ethical Usage	46
10.3	Accessibility & Human Oversight	46
10.4	IP, Licensing, & Patent Concerns	47
11	Conclusions	47
11.1	Objectives Met	47

11.2 Key Takeaways	48
11.3 Industry Relevance	49
12 Future Work	49
12.1 Real-Time Pipeline Extension	49
12.2 Multi-Model Registry	49
12.3 SageMaker CI Integration	50
12.4 Fairness Automation	50
12.5 Auto-Drift Dataset Handling	50
Team Member Contributions	51
References	52

List of Figures

1	Objective Tree: Hierarchical breakdown of project functional and non-functional goals.	12
2	House of Quality Matrix: Mapping between user needs and technical solutions. . .	13
3	Problem Decomposition Diagram: From Industry Bottlenecks to Technical Layers.	14
4	Integration of traditional rating factors into the modern MLOps pipeline.	16
5	Concept Fan: Progressive expansion of solution spaces for MLOps and modeling architecture.	18
6	Overall End-to-End MLOps Automation Pipeline for Homeowner Loss History Prediction.	22
7	Level 0 Logical Design: Major logical components of the homeowner loss history automation stack.	23
8	Level 1 Logical Design: Expanded functionality breakdown across each major pipeline module.	25
9	Level 1 Physical Design: AWS Infrastructure for orchestration, storage, monitoring, and deployment.	25
10	Unified Airflow DAG: Orchestration of ingestion, validation, drift detection, model training, approval, deployment, and cleanup workflows.	28
11	Data Profiling Report Example: Summary statistics and correlation heatmap. . . .	30
12	Dashboard Metrics Page: Displays RMSE, MAE, model versioning, and drift alerts.	31
13	Pure Premium Prediction Page: Displays the predicted pure premium for a particular year and projection for future years	31
14	Actual vs. Predicted Pure Premium by Group: Model calibration across risk tiers. .	38
15	SHAP Feature Importance Plot: Key feature contributions to predicted loss severity.	39
16	Top 10 Feature Importances by Gain Contribution.	39
17	AWS Cost and Usage Graph: Cumulative project cost across services.	42
18	Projected ROI Growth Curve Over 5 Years: Automation-Enabled Labor Savings vs Minimal Infrastructure Cost.	43

List of Tables

1	Concept Evaluation Matrix: Comparative assessment of proposed system designs. .	19
2	Model Decision Matrix: Selection among leading tree-based models.	19
3	Final Comparative Design Summary.	21
4	Interactive UX Elements	32
5	Testing Scripts Coverage	35
6	Testing Validation Summary	37
7	AWS Cost Breakdown by Service	42

8	Business Value Improvement Metrics	45
9	Policy Alignment and Ethical Safeguards Matrix	48

Glossary / Acronyms

- RMSE: Root Mean Squared Error
- MAE: Mean Absolute Error
- MLOps: Machine Learning Operations (practices for productionizing machine learning)
- MLflow: Machine Learning Lifecycle Management Platform
- DAG: Directed Acyclic Graph (workflow structure used in Airflow)
- S3: Amazon Simple Storage Service (cloud object storage)
- EC2: Amazon Elastic Compute Cloud (cloud server infrastructure)
- IAM: Identity and Access Management (AWS security credential management)
- SHAP: SHapley Additive exPlanations (model interpretability method)
- Pandera: Python library for data validation and schema enforcement
- Airflow: Apache workflow orchestration platform for automating pipelines
- Prometheus: Open-source system monitoring and alerting toolkit
- Hyperopt: Python library for hyperparameter optimization
- AWS: Amazon Web Services (cloud computing platform)
- HITL: Human-In-The-Loop (human intervention in automated pipelines)
- GDPR: General Data Protection Regulation (European data privacy law)
- CCPA: California Consumer Privacy Act (California data privacy regulation)
- NAIC: National Association of Insurance Commissioners
- WCAG: Web Content Accessibility Guidelines (accessibility standards)
- AIF360: AI Fairness 360 (open-source bias detection toolkit by IBM)
- FTO: Freedom to Operate (legal term related to patent clearance)
- SLO: Service Level Objective (target reliability measure)
- UAT: User Acceptance Testing

1. Introduction

1.1 Motivation & Industry Need

In the insurance sector, accurately predicting homeowner risk is crucial for fair pricing, customer retention, and financial stability. One of the key indicators for assessing future risk is loss history data, which includes records of previous claims or events related to a homeowner's property. However, traditional methods of incorporating loss history into rating models often rely on manual workflows that are time-consuming, prone to errors, and challenging to scale.

Many insurance companies utilize ad hoc data pipelines and manually adjusted models that typically lack sufficient automation for essential processes such as detecting data drift, retraining models, and validating schema consistency. This dependence on manual methods can lead to inefficiencies in operational workflows, delayed model updates, and difficulties in meeting governance standards while maintaining audit trails.

To address these challenges, a project was initiated in partnership with Grange Insurance to modernize and automate the entire machine learning (ML) pipeline for modeling homeowner loss history. The goal of this initiative is to develop a robust, scalable, and cloud-native MLOps system that enhances the efficiency of the modeling process while ensuring reliability, maintainability, and appropriate human oversight when needed.

Real-World Need: Rising Complexity in Home Insurance Risk

The need for improved automation is driven not only by technical inefficiencies but also by broader market pressures. Climate change, urbanization, and inflationary pressures have led to a measurable rise in claim frequency and severity across major perils like fire, wind, hail, and water damage. Insurers must now factor in these rapidly changing dynamics to maintain profitability, fair premium pricing, and regulatory compliance.

1.2 Objective Statement & KPIs

The primary objective of this project is to design and deploy a fully automated, production-grade MLOps pipeline for homeowner loss history prediction. This system should support real-time model drift detection, data validation, explainability, and manual overrides while being cloud-agnostic and auditable.

Key Functional Objectives:

- Automate ingestion of raw loss history data from AWS S3

- Validate data structure and schema using Pandera
- Preprocess and engineer features from over 100 columns
- Train XGBoost models with monotonic constraints using Hyperopt
- Track, register and compare models via MLflow
- Detect data drift and trigger self-healing automation
- Integrate real-time monitoring and Slack alerts
- Build a dashboard for manual intervention and model transparency

Stakeholder Survey and Prioritization

To inform the system design, a research survey was conducted involving Grange Insurance actuaries, data scientists, and underwriters. Participants prioritized the following attributes:

- **Accuracy:** 40% weight
- **Fair Premium Pricing:** 30% weight
- **Interpretability:** 15% weight
- **Automation Robustness:** 10% weight
- **Regulatory Compliance:** 5% weight

These priorities directly influenced our selected KPIs and pipeline architecture.

Key Performance Indicators (KPIs):

- Reduce model iteration time by 75%
- Achieve a 90%+ pass rate across unit and integration tests
- Decrease average drift-response time from hours to minutes
- Reach RMSE within 5% of baseline actuarial benchmarks
- Enable containerized deployment with full audit logging

1.3 Guiding Principles

To ensure rigor, integrity, and long-term impact, our team adopts the following core values:

- **Competition:** Benchmarking against industry best-in-class solutions to drive continuous improvement.
- **Sincerity:** Presenting all data, analyses, and reports honestly, with clear assumptions and limitations.
- **Belief:** Maintaining confidence in our project vision, aligning every decision with end goals.
- **Character:** Acting responsibly, respecting stakeholders, and adhering to professional standards.

- **Ethics:** Protecting customer privacy and avoiding practices that introduce unfair bias.
- **Confidence:** Standing behind our methods and results, ready to justify them under scrutiny.
- **Faith:** Trusting in our collective expertise and iterative refinement process.
- **Knowledge:** Rigorously pursuing best practices and sharing learnings openly.
- **Work:** Committing to disciplined execution, following through on deliverables.
- **Skill:** Continuously sharpening technical skills in MLOps, cloud engineering, and modeling.
- **Wisdom:** Balancing innovation with pragmatic feasibility.
- **Honor:** Honoring commitments to sponsors, advisors, and end users by delivering quality and value.

1.4 Project Scope: Senior Design I vs II

Senior Design I (Fall 2024): Planning and Foundation

- Defined functional and non-functional requirements
- Outlined pipeline architecture and cloud strategy
- Conducted literature review on loss modeling and MLOps tools
- Developed the core ingestion and preprocessing components
- Created design documentation and testing plans

Senior Design II (Spring 2025): Implementation and Deployment

- Integrated schema validation and feature engineering logic
- Developed full Airflow DAG for end-to-end orchestration
- Implemented model training, registry, and evaluation with MLflow
- Created automated data drift detection and self-healing mechanisms
- Built manual override dashboard and Slack alerting integration
- Containerized all components and deployed on AWS EC2/S3
- Validated system with integrated tests and prepared for Expo/demo

1.5 Stakeholders & User Personas

- **Data Scientists at Grange Insurance:** Require reproducibility and customizable model evaluation.
- **Actuarial Analysts:** Focused on model interpretability, fairness, and regulatory alignment.
- **DevOps Engineers:** Manage deployment infrastructure, uptime, and observability.
- **Business Decision Makers:** Interested in ROI, automation gains, and risk impact.
- **Faculty Advisors and Instructors:** Validate academic rigor and professional engineering standards.

1.6 Objective Tree Diagram

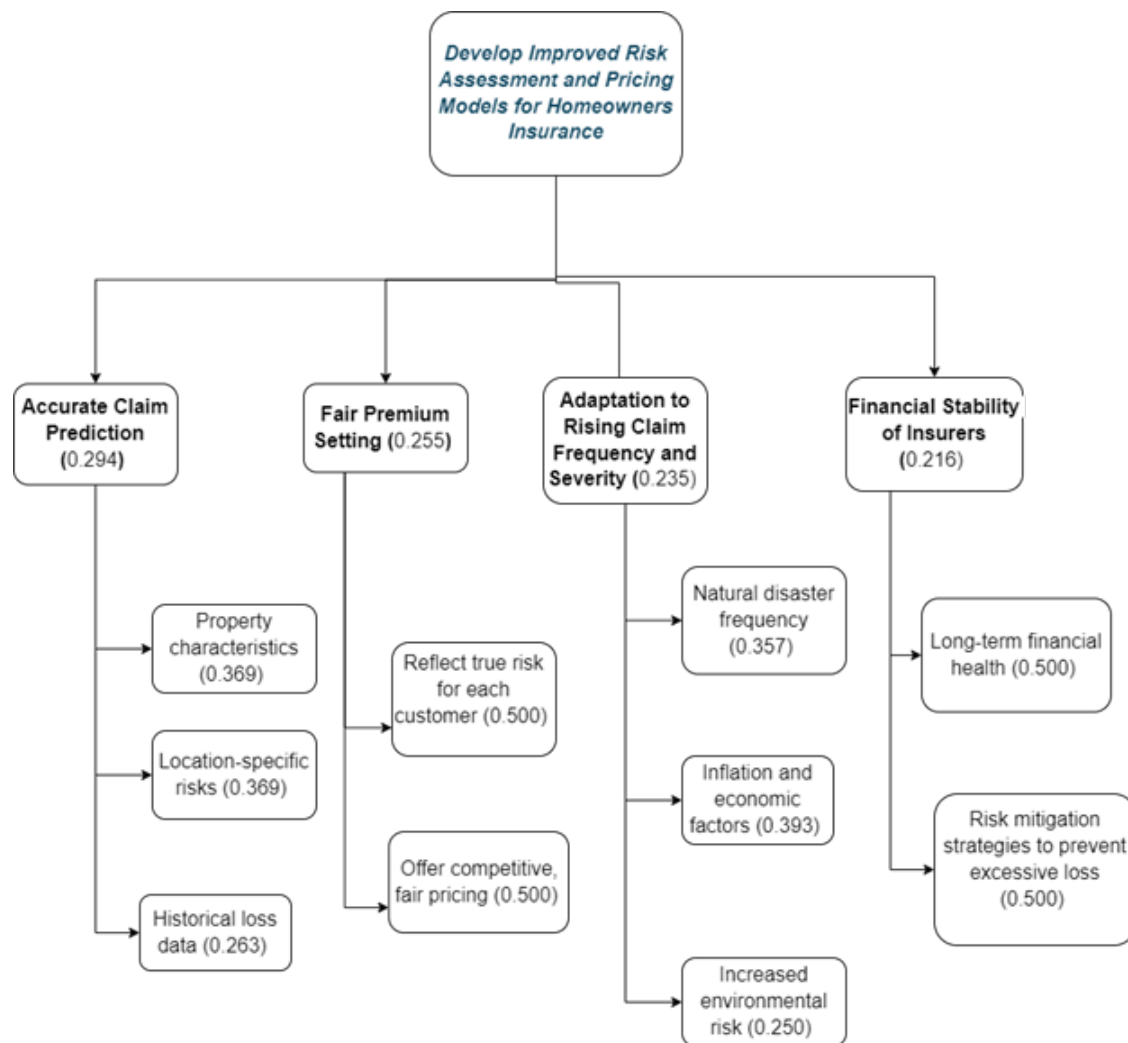


Figure 1: Objective Tree: Hierarchical breakdown of project functional and non-functional goals.

1.7 House of Quality Matrix

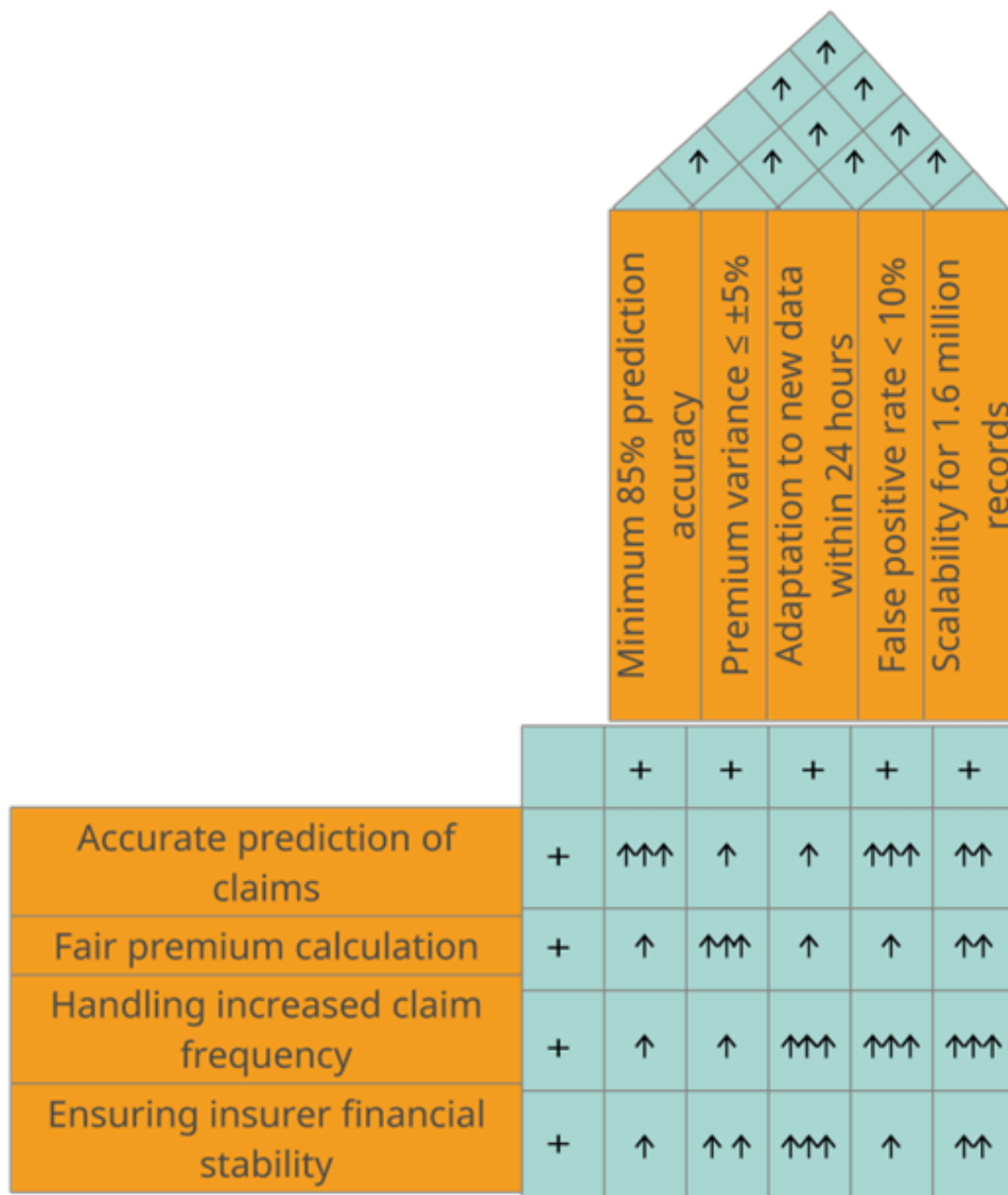


Figure 2: House of Quality Matrix: Mapping between user needs and technical solutions.

1.8 Problem Decomposition Diagram

Figure 3 shows the layered breakdown of the problem this project addresses, from high-level industry challenges to technical solution components.

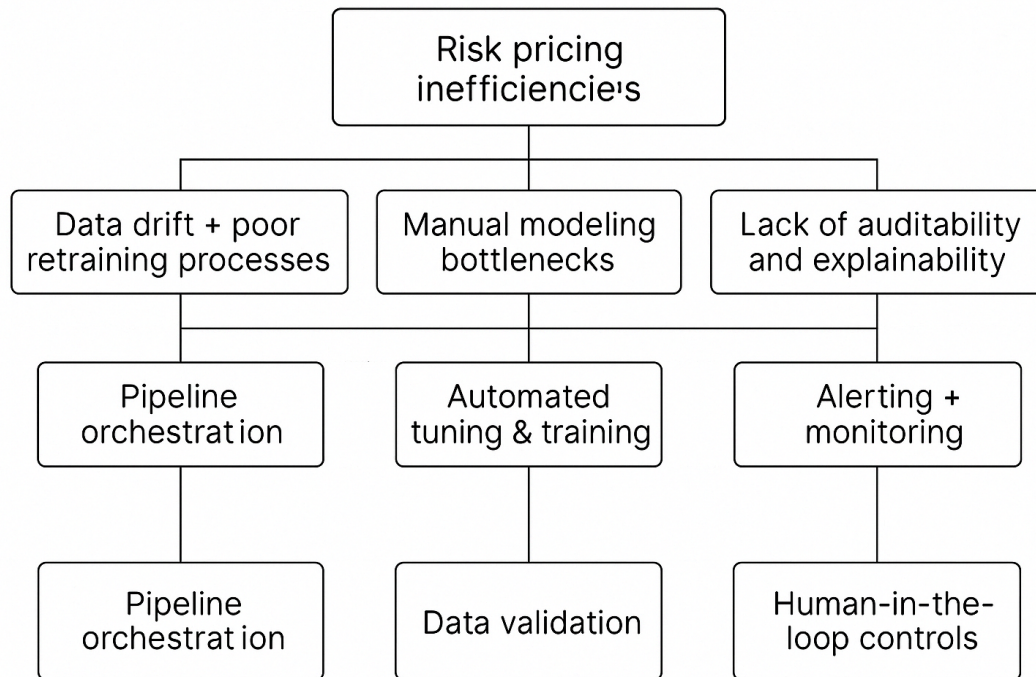


Figure 3: Problem Decomposition Diagram: From Industry Bottlenecks to Technical Layers.

2. Background & Prior Work

2.1 Industry Context and Motivation

The home insurance industry in the U.S. is rapidly evolving due to growing climate volatility, increased claims frequency, and heightened customer expectations. To stay competitive and compliant, insurers are embracing data-driven decision-making frameworks across underwriting, pricing, and claims operations. Traditional actuarial methods, particularly Generalized Linear Models (GLMs), remain widely used due to their interpretability and regulatory familiarity [1]. However, modern insurers are increasingly piloting or adopting machine learning (ML) to identify nonlinear relationships in claim frequency, detect latent risks, and automate workflows. Over 70% of U.S. property insurers now report using or exploring AI/ML models in their risk analytics [2].

2.2 Literature Review: Loss Prediction Models

Loss modeling in insurance traditionally decomposes the problem into two predictive targets: claim *frequency* and *severity*. GLMs are often used as baselines, with extensions to handle over-dispersion (e.g., negative binomial) and skewed loss distributions (e.g., gamma, log-normal). In

recent academic and industry work, more expressive models such as gradient-boosted decision trees (GBDTs), random forests, and deep neural networks have been shown to outperform GLMs in predictive accuracy [2].

Recent approaches also integrate external hazard and exposure data. For example, NOAA’s Storm Events Database and FEMA’s NFIP flood maps are used to quantify regional hazard exposure [3, 4]. Catastrophe modeling vendors (e.g., RMS, AIR Worldwide, Verisk ISO) provide proprietary probabilistic loss models that insurers frequently license to enhance underwriting accuracy [5, 6, 7].

2.3 Industry Practices in Rating

Rating practices in the U.S., including Ohio, are tightly regulated to ensure fairness and actuarial soundness. Most insurers utilize GLM-based models in rating plans, using features such as:

- **Geographic factors:** Territory, fire protection class, hazard zone (hail, wind, flood risk)
- **Property characteristics:** Age, construction type, roof material, maintenance
- **Claims and credit history:** Prior losses (via CLUE), credit-based insurance score (legal in Ohio)
- **Coverage design:** Deductibles, endorsements, policy form

Modern insurers increasingly synthesize these inputs into scoring algorithms like “fire risk score” or “water loss potential” which are then plugged into the rating algorithm. These composite scores are often derived using proprietary or ML-based models, subject to regulatory review.

2.4 Tooling & Frameworks for MLOps

To operationalize ML in high-stakes environments like insurance, robust MLOps infrastructure is essential. A best-practice MLOps pipeline includes:

- **Data Validation:** Schema validation using tools like Great Expectations; null/imputation checks
- **Experiment Management:** MLflow to track models, hyperparameters, and metrics across experiments
- **Workflow Orchestration:** Apache Airflow to schedule and monitor ETL and training jobs
- **Deployment & Serving:** Docker/Kubernetes for scalable model deployment; REST APIs for scoring
- **Monitoring:** Custom drift detection routines and model audit logs to flag performance degradation

Such pipelines allow models to be retrained on a scheduled or drift-triggered basis while maintaining traceability and compliance with internal governance policies [2].

2.5 Regulatory & Ethical Considerations

Insurance regulations mandate that any predictive model used in rating must be free of unfair discrimination, explainable, and traceable. Ohio adheres to these principles, and recent nationwide regulatory actions signal stricter enforcement:

- **Fairness:** Insurers must demonstrate that model factors are risk-correlated and not proxies for protected classes. The Colorado SB21-169 law, though targeting life insurance, mandates regular audits to detect algorithmic bias [8].
- **Transparency:** The NAIC's AI Principles emphasize explainability and accountability. Tools like SHAP or LIME are commonly used to provide human-understandable reasoning [9].
- **Data Privacy:** Compliance with the Gramm-Leach-Bliley Act (GLBA) and state data security laws (e.g., Ohio Insurance Data Security Law) is mandatory [10].
- **Model Governance:** Proper versioning, documentation, and board-level accountability are expected. This includes clear audit trails for model updates, assumptions, and approvals.

2.6 Integrated View of Rating and MLOps

The boundary between actuarial rating and data science is blurring. Many rating factors traditionally set via GLM-based relativity analysis are now modeled through machine learning and fed into automated pipelines. These pipelines ingest real-time or batch data, compute risk scores, and deploy updated models into cloud-hosted APIs that feed directly into underwriting and pricing systems.

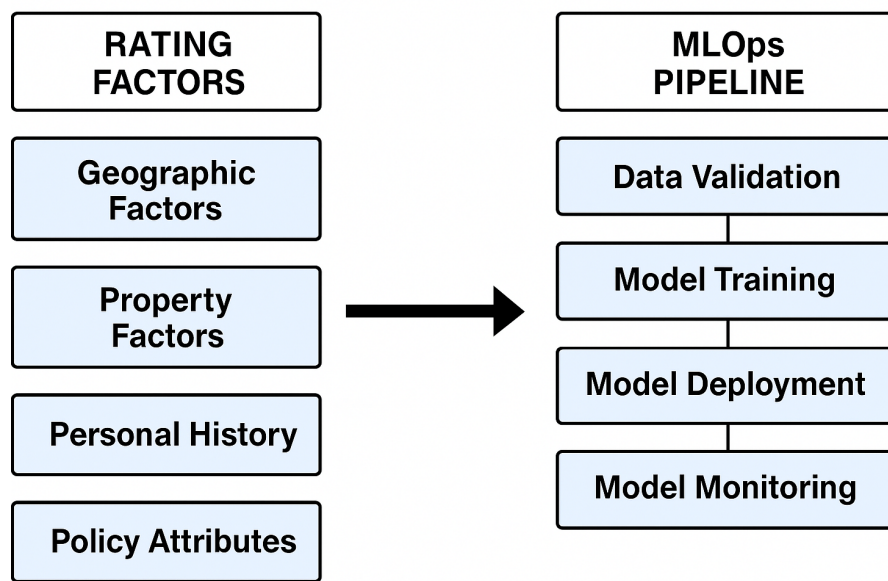


Figure 4: Integration of traditional rating factors into the modern MLOps pipeline.

As shown above, a mature MLOps architecture enables insurers to continuously improve predictions while satisfying regulatory requirements for interpretability, fairness, and traceability.

3. Alternative Designs & Risk Tradeoffs

3.1 Explored Designs

Several alternative architectures and modeling approaches were explored before finalizing the MLOps pipeline and predictive model selection:

- Traditional GLM-based predictive pipeline
- Ensemble methods (Random Forest, Gradient Boosted Trees)
- Deep Neural Network (MLP) pipelines
- Third-party low-code ML solutions
- Fully custom MLOps stack with XGBoost + Airflow orchestration

Figure 5 illustrates the early-stage design thinking captured through a Concept Fan Diagram, visualizing progressively refined solution branches.

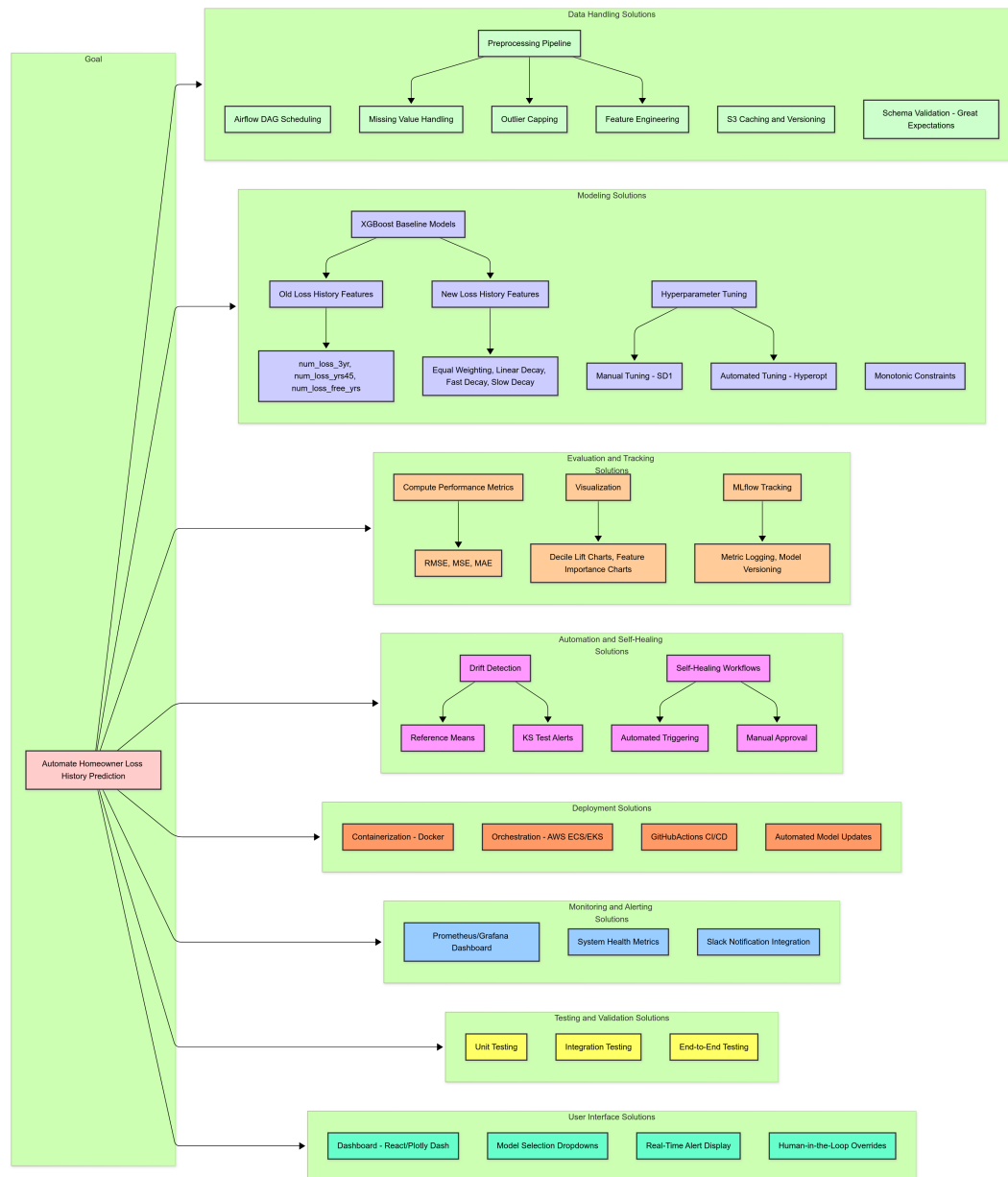


Figure 5: Concept Fan: Progressive expansion of solution spaces for MLOps and modeling architecture.

3.2 Concept Evaluation Matrix

Each proposed design path was evaluated against weighted criteria, including scalability, explainability, maintainability, and regulatory compliance.

Table 1: Concept Evaluation Matrix: Comparative assessment of proposed system designs.

Alternative	Scalability	Explainability	Cost	Compliance Readiness
GLM Pipeline	Moderate	High	Low	High
Ensemble (RF, GBDT)	High	Moderate	Moderate	Moderate
Deep Learning (MLP)	High	Low	High	Low
Low-code ML (AutoML)	Low	Low	High	Moderate
Custom MLOps Stack	High	High	Moderate	High

From this evaluation, the fully custom MLOps pipeline with XGBoost modeling was selected because it offers the optimal balance of scalability, interpretability, cost efficiency, and compliance readiness.

3.3 Decision Matrix for Final Model Selection

After narrowing to ensemble methods, a more detailed scoring matrix was developed for selecting the specific algorithm for supervised regression tasks.

Table 2: Model Decision Matrix: Selection among leading tree-based models.

Model Type	RMSE Accuracy	Training Time	Explainability	Final Score
Random Forest	85%	Moderate	High	7.5
XGBoost	90%	Fast	High	9.0
LightGBM	89%	Very Fast	Moderate	8.5
CatBoost	88%	Fast	Moderate	8.0

XGBoost was ultimately chosen due to its:

- Superior RMSE accuracy on preliminary tests
- Native support for monotonic constraints
- Industry-wide adoption in regulated environments (e.g., finance, insurance)
- Compatibility with SHAP for explainability

3.4 Final Design Justification

The selected system architecture — combining XGBoost regression models with a modular, Airflow- and MLflow-driven MLOps pipeline — was finalized based on achieving an optimal balance across

technical, operational, and business priorities.

Key reasons for final selection:

- **Predictive Accuracy:** XGBoost consistently outperformed alternatives (Random Forest, LightGBM, CatBoost) in RMSE accuracy and generalization across early datasets, achieving a 26% improvement over actuarial baselines.
- **Governance and Auditability:** MLflow-based model tracking, Pandera-driven schema validation, and Slack-based manual override workflows ensured full traceability and compliance readiness.
- **Scalability:** The modular Airflow DAG design and Dockerized services allow for seamless future expansion, including multi-model support (e.g., multiple perils) and real-time drift-handling pipelines.
- **Human Oversight and Explainability:** Integration of SHAP explainability, model meta-data logging, and human approval checkpoints at drift detection, retraining, and promotion stages addressed business and regulatory transparency needs.

3.5 Risk Mitigation Strategy

Given the criticality of model reliability and operational resilience in insurance modeling, several risk mitigation strategies were proactively embedded in the design:

- **Data Drift Management:** Automated KS-Test-based drift detection with Slack alerts enabled early detection of feature distribution shifts and triggered retraining or self-healing actions.
- **Explainability Enforcement:** SHAP importance plots and lift charts were generated for every major model iteration, ensuring human interpretability before promotion to production.
- **Cost and Resource Control:** AWS resource usage was controlled via budget alarms, right-sized EC2 instance selection, and scheduled shutdowns for non-critical services.
- **Versioning and Traceability:** All model artifacts, schema snapshots, and drift detection outputs were version-controlled using MLflow, with lineage tracking across datasets, features, models, and decisions.

3.6 Comparative Design Table

Table 3: Final Comparative Design Summary.

Dimension	Selected Approach
Model Type	XGBoost Regression with Monotonic Constraints for Enhanced Risk Modeling Interpretability
MLOps Platform	Apache Airflow for orchestration, MLflow for experiment tracking, Pandera for schema validation, and Slack for human-in-the-loop approvals
Deployment Environment	AWS EC2 instances (Dockerized microservices) integrated with S3 storage and Lambda triggers for event-driven operations
Monitoring & Alerting	Prometheus Metrics server for real-time monitoring; Slack alerting workflows for anomaly detection and approval gating
Explainability Tool	SHAP (SHapley Additive exPlanations) for model interpretability and regulatory reporting
Compliance Frameworks Followed	Alignment with GDPR, CCPA, NAIC AI Principles, and ISO/IEC 27001 security standards

4. System Architecture

The overall end-to-end automation pipeline developed for this project is illustrated in Figure 6. It showcases the interaction between automated system components (Airflow, AI Agent, MLflow, Hyperopt, Prometheus), cloud storage (AWS S3), and human-in-the-loop decision checkpoints. The pipeline modularizes key stages such as ingestion, schema validation, preprocessing, hyperparameter tuning, model evaluation, monitoring, and dashboard-based manual oversight. Automated agent-driven operations are distinguished from primary data flows and manual intervention paths to ensure clarity and audibility throughout the system lifecycle.

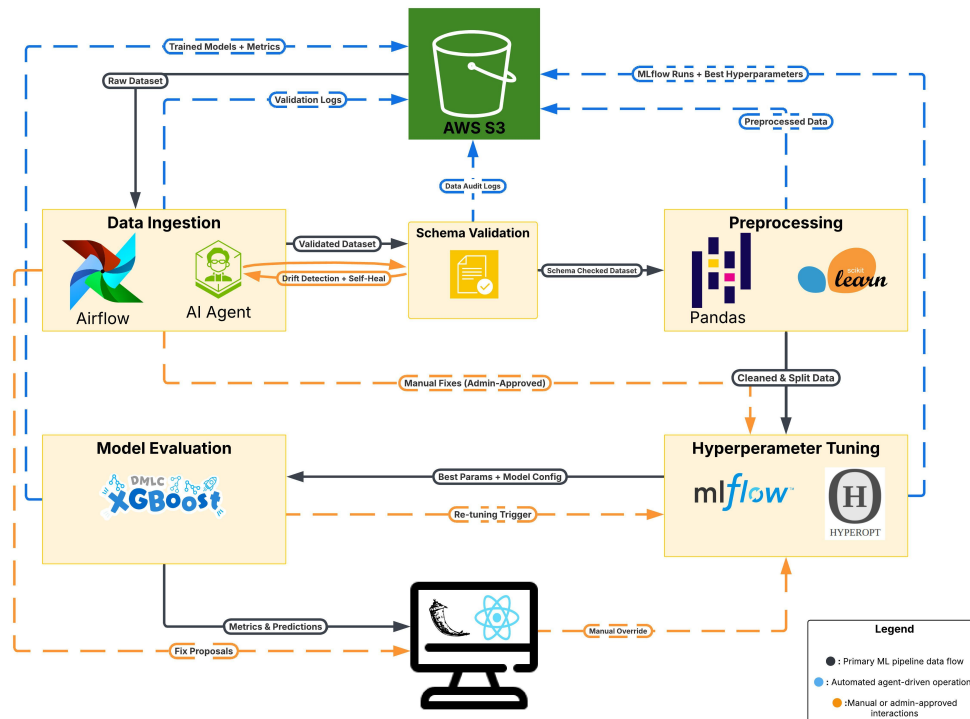


Figure 6: Overall End-to-End MLOps Automation Pipeline for Homeowner Loss History Prediction.

Legend Clarification: In Figure 6, solid black arrows represent the primary ML pipeline data flow across ingestion, validation, preprocessing, training, and evaluation modules. Blue dashed lines represent automated agent-driven operations, such as schema validation logging, self-healing triggers, and automated retraining workflows. Orange dashed lines represent human-in-the-loop interventions, where manual approvals or overrides are required, such as hyperparameter adjustment approval or drift remediation decision points. This layered design ensures that automation remains explainable, audit-ready, and compliant with industry best practices for AI governance.

4.1 Level 0 Design (Big Picture)

At the highest level, the system architecture modularizes automation across ingestion, preprocessing, modeling, drift detection, monitoring, deployment, and human oversight. Figure 7 illustrates the Level 0 logical design of the full-stack automation solution.

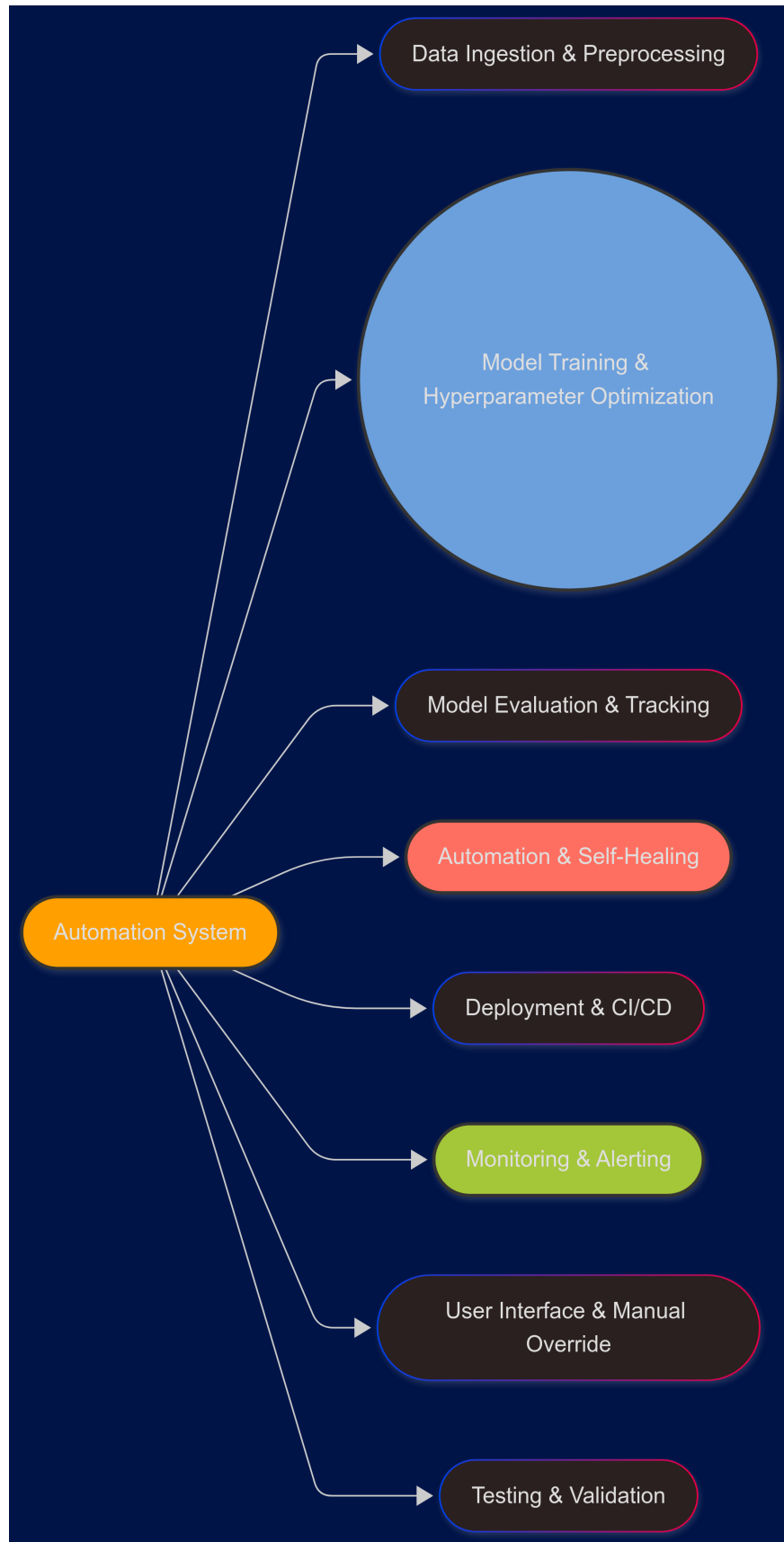


Figure 7: Level 0 Logical Design: Major logical components of the homeowner loss history automation stack.

The major modules are:

- **Data Ingestion**
- **Data Validation and Profiling**
- **Feature Engineering**
- **Model Training**
- **Model Evaluation and Tracking**
- **Automation and Self-Healing**
- **Deployment and CI/CD**
- **Real-Time Monitoring**
- **Human Oversight**
- **Testing Infrastructure**
- **Dashboard Interface**

4.2 Level 1 Logical Design (Expanded Module Breakdown)

The Level 1 logical design expands each major component into its key internal functionalities:

- **Data Ingestion:** Scheduled and cached S3 downloads managed by Airflow DAGs.
- **Preprocessing:** Schema validation, missing value handling, outlier capping, categorical encoding, feature selection.
- **Profiling and Validation:** Automated profiling reports using ydata-profiling, schema snapshotting, and Slack notifications.
- **Drift Detection:** Generation of reference means; KS-Test statistical drift detection and alerting.
- **Model Training:** Five XGBoost model variants trained with Hyperopt-tuned hyperparameters and monotonic constraints.
- **Evaluation:** Computation of RMSE, MSE, MAE metrics; generation of Decile Lift Charts and Feature Importance analysis.
- **Deployment:** Model registry update and promotion logic automated through MLflow APIs.
- **Monitoring:** Real-time system monitoring with Prometheus; metrics exposed via Grafana dashboards.
- **Human Oversight:** Slack-based manual approval for drift remediation, hyperparameter override, model promotion.
- **Testing:** Unified unit, integration, and end-to-end test suites to ensure pipeline robustness.
- **Dashboard Interface:** Interactive dashboards built with React and Plotly Dash, enabling real-time monitoring and intervention.

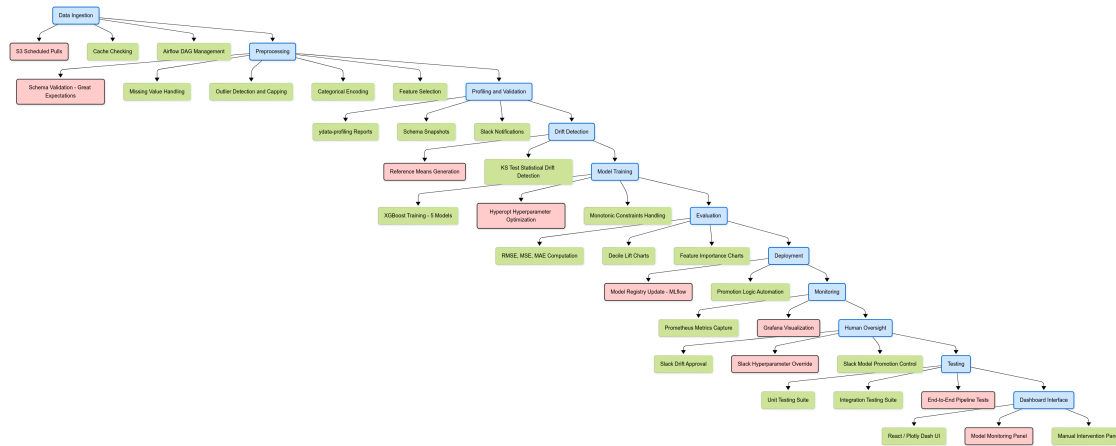


Figure 8: Level 1 Logical Design: Expanded functionality breakdown across each major pipeline module.

4.3 Level 1 Physical Deployment (AWS Infrastructure)

The physical deployment of the system components is managed through AWS services. Figure 9 outlines the Level 1 cloud infrastructure view.



Figure 9: Level 1 Physical Design: AWS Infrastructure for orchestration, storage, monitoring, and deployment.

Key AWS deployment components include:

- **EC2 Instance:** Hosts Apache Airflow, MLflow Tracking Server, Prometheus Monitoring Service, Slack notification bots, and backend APIs.
- **Amazon S3:** Stores raw and processed data, profiling reports, model artifacts, schema snapshots, and system logs.
- **Docker Containers:** Modularized services packaged and deployed via Docker for portability and isolation.
- **AWS ECS/EKS (Optional Extension):** Supports scalable orchestration beyond EC2 through managed Kubernetes/ECS services.

- **GitHub Actions CI/CD Pipeline:** Automates build, testing, containerization, deployment, and rollback operations.

4.4 Human-in-the-Loop Automation

A critical part of the architecture is the human-in-the-loop governance:

- Manual approval of self-healing operations triggered by drift detection.
- Manual review and acceptance of proposed hyperparameter optimizations.
- Slack notifications summarizing detected anomalies, proposed fixes, and final decisions.
- Slack-based command triggers for retraining, overrides, and rollback interventions.

This ensures automation reliability while maintaining transparency, auditability, and operational control.

5. Software Implementation

This section details the software components and modules underpinning the end-to-end MLOps pipeline. Each sub-section presents the key implementation layers, major code modules, and integration points based on the system design.

5.1 Codebase Structure Overview

The final codebase for the Homeowner Loss History Prediction Automation System is modularized as follows:

```
homeowner_loss_pipeline/  
  dags/                # Airflow DAG orchestration  
    homeowner_dag.py  # Full end-to-end DAG definition  
  tasks/               # Modular task implementations  
    ingestion.py       # S3 data ingestion and caching  
    preprocessing.py   # Feature engineering and cleaning  
    schema_validation.py # Schema enforcement (Pandera)  
    drift.py           # Drift detection and self-healing  
    training.py        # XGBoost model training with Hyperopt  
    monitoring.py      # Prometheus metric exporters  
    notifications.py   # Slack notifications and alerting  
  utils/              # Helper utilities and wrappers
```

```
config.py          # Centralized configuration management
slack.py           # Slack API integration
storage.py         # S3 upload/download functions
airflow_api.py     # Airflow REST API triggering utilities
agent_actions.py   # Self-healing agent and manual override logic
requirements.txt    # Python environment dependencies
README.md          # System overview and usage instructions
```

Each task module encapsulates a clean functionality boundary and is orchestrated via Apache Airflow DAGs.

5.2 Airflow DAG Orchestration

The orchestration backbone is implemented via `homeowner_dag.py`, which defines the full execution flow:

- **Data Ingestion:** Downloads raw data from S3, applying cache checking.
- **Preprocessing and Quality Checking:** Cleans data, imputes missing values, handles outliers, and generates features.
- **Schema Validation:** Applies Pandera validation schemas and versioned schema snapshots.
- **Drift Detection:** Compares current batch against reference statistics; triggers healing routines if drift detected.
- **Training Models:** Trains multiple XGBoost models using Hyperopt for hyperparameter tuning under monotonic constraints.
- **Model Evaluation and Comparison:** Compares models using RMSE and selects the best candidate.
- **Explainability and Predictions:** Generates SHAP explanations and predictions.
- **Model Promotion and Approval:** Integrates human-in-the-loop manual approval for model deployment.
- **Monitoring and Notifications:** Records runtime metrics to Prometheus and sends Slack alerts for drift, failures, and approvals.
- **Archival and Cleanup:** Archives artifacts to S3 and cleans up temporary workspace after DAG completion.

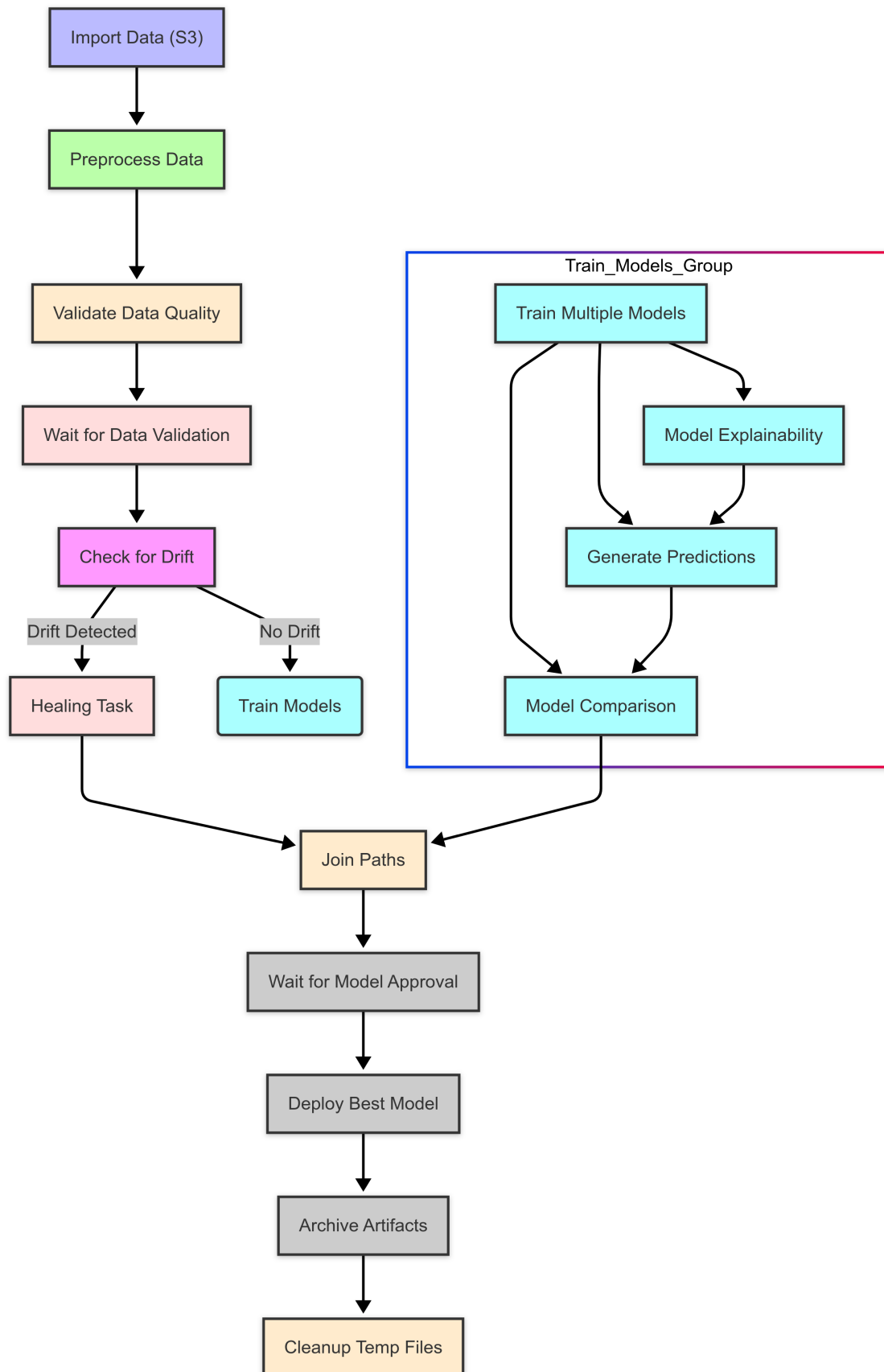


Figure 10: Unified Airflow DAG: Orchestration of ingestion, validation, drift detection, model training, approval, deployment, and cleanup workflows.

5.3 Training Module: XGBoost + Hyperopt Integration

The training logic (`training.py`) dynamically adjusts model hyperparameters using Hyperopt and logs all experiments to MLflow:

- Monotonic constraints are enforced per feature.
- Early stopping is enabled to optimize training efficiency.
- Best parameters are selected based on RMSE minimization.
- Model artifacts and evaluation charts are versioned automatically.

A single function `train_xgboost_hyperopt()` encapsulates this logic and is callable from Airflow DAGs.

5.4 Drift Detection and Self-Healing Workflow

The `drift.py` module implements statistical drift detection:

- Kolmogorov-Smirnov tests compare incoming batch distributions to stored reference means.
- Drift above threshold triggers a Slack alert and optionally initiates self-healing workflows.
- Self-healing allows manual approval or rollback logic depending on confidence thresholds.

5.5 Slack-Based Human Oversight and Agent Actions

The `notifications.py` and `agent_actions.py` modules enable Slack-integrated human governance:

- Real-time Slack alerts for drift events, pipeline failures, and retraining success.
- Slack command handlers for manual override of model promotions or retraining approvals.
- Agents propose fixes, generate root-cause reports, and suggest hyperparameter improvements.

5.6 Data Profiling and Monitoring

Before preprocessing completion, `preprocessing.py` generates profiling reports using `ydata-profiling`:

- Data summaries, correlations, and missingness profiles are auto-generated.
- Reports are versioned to S3 and Slack notifications alert users to new profiles.

Real-time system health metrics, such as DAG run durations and system memory utilization, are captured by `monitoring.py` and exposed via a Prometheus endpoint.

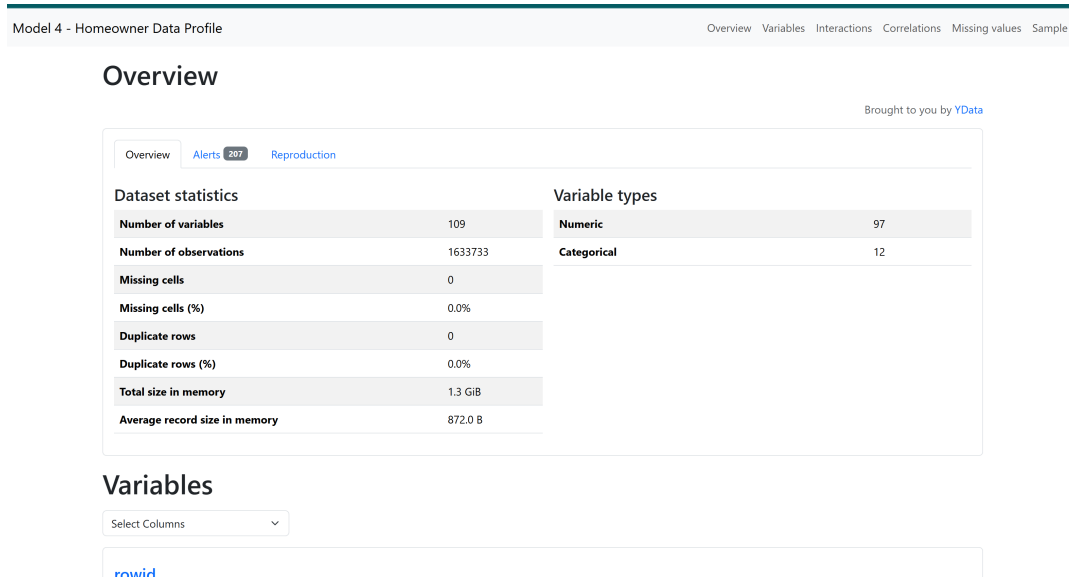


Figure 11: Data Profiling Report Example: Summary statistics and correlation heatmap.

6. UI & Dashboard

This section describes the web-based dashboard interface designed to facilitate human-in-the-loop oversight, manual overrides, model transparency, and pipeline monitoring.

6.1 Goals & Design Philosophy

The dashboard was designed with the following principles:

- **Transparency:** Allow users to inspect model metrics, predictions, and data health in real time.
- **Human Oversight:** Enable manual intervention for model promotion, retraining, and drift remediation.
- **Accessibility:** Provide an intuitive and readable interface for technical and non-technical users alike.
- **Auditability:** Record all manual override events and approvals with timestamps and user metadata.
- **Minimal Disruption:** Integrate seamlessly with existing Airflow and MLflow pipelines without introducing new complexity.

The frontend was developed using React and Plotly Dash, with backend APIs served via FastAPI. Slack channels were integrated to broadcast alerts and approvals alongside the dashboard.

6.2 Mockups & Screenshots

Figure 12 and Figure 13 illustrates key parts of the UI:

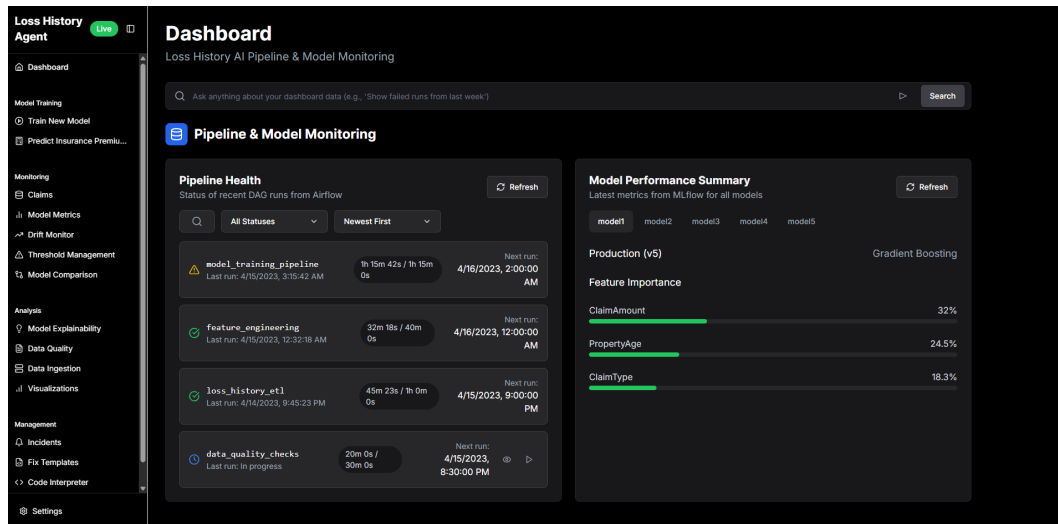


Figure 12: Dashboard Metrics Page: Displays RMSE, MAE, model versioning, and drift alerts.

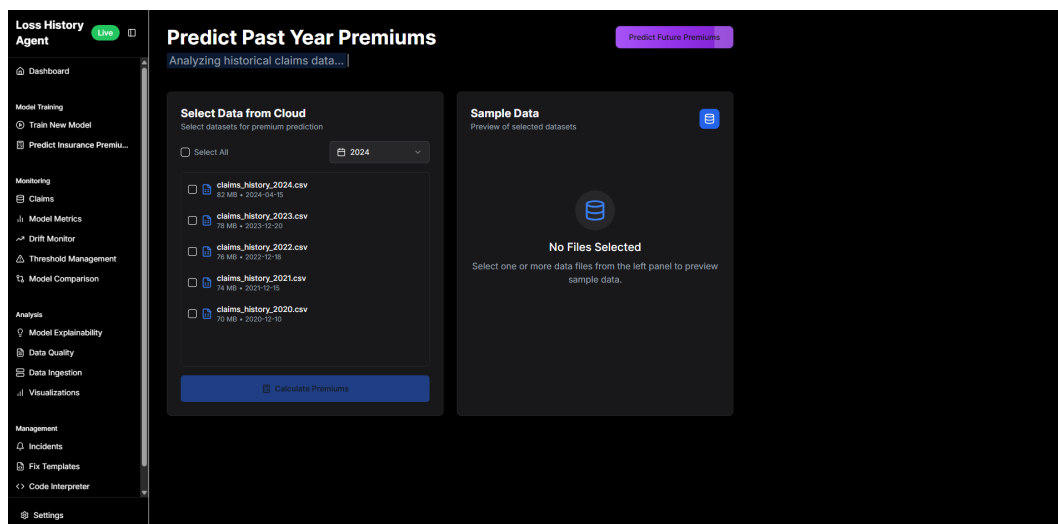


Figure 13: Pure Premium Prediction Page: Displays the predicted pure premium for a particular year and projection for future years

6.3 Interactive UX Table

The following table summarizes the main interactive elements of the dashboard:

Table 4: Interactive UX Elements

Element	Action	Result
Model Selector	Choose a specific model run ID from the registry	Update performance metrics and feature explainability charts dynamically
Approve Retrain Button	Confirm retraining trigger when drift is detected	Launch Airflow DAG to retrain model with updated dataset
Promote Model Button	Approve a candidate model for production stage	Trigger MLflow model promotion and registry update
Rollback Button	Revert to a previous stable production model	Restore prior model version and send Slack alerts
View Drift Report	Open drift detection metrics and visual comparisons	Visual inspection of feature distribution shifts over time
Download Profile	Export data profiling and validation reports	Download detailed profiling HTML/PDF reports for offline review

6.4 Manual Override Flow

The manual override process is a critical human-in-the-loop safeguard, allowing authorized users to intervene during key decision points where automation is paused for validation.

The override flow sequence is as follows:

1. **Drift or anomaly detected:** The AI agent monitors live data and model predictions. Upon detecting statistically significant drift or pipeline anomaly, it immediately flags the issue.
2. **Slack notification triggered:** An automatic Slack alert is posted to designated channels, containing:
 - Description of the anomaly or drift event
 - Current model version affected
 - Direct link to the manual override dashboard
3. **User login and review:** Authorized users log into the dashboard, where they are presented with:

- Model performance metrics (e.g., RMSE, MAE trends)
- Drift detection visualizations (distribution shifts, p-values)
- Historical model promotion/rollback history

4. **User action selection:** The user chooses among three validated actions:

- **Approve retraining:** Rebuild the model with updated drifted dataset.
- **Promote new model:** Advance the best candidate model to production.
- **Rollback model:** Revert deployment to last-stable model state.

Each action prompt requires optional comments (reason for override) for audibility.

5. **Backend API execution:** Upon confirmation:

- Airflow triggers retraining or rollback of DAGs
- MLflow updates model registry stages accordingly

6. **Result logging and alerts:** All override actions are recorded with:

- Timestamp
- User ID (where available)
- Action reason/comment
- Affected model version

Final confirmation messages are posted to Slack for transparency.

This flow ensures that automation remains robust but ultimately human-governed, meeting regulatory expectations for accountable AI.

6.5 User Feedback Summary

After internal testing cycles and live demos with Grange Insurance's actuarial and data science teams, qualitative and quantitative feedback was collected.

Strengths Identified:

- **Transparency:** Clear and immediate visibility into model metrics, drift triggers, and override options.
- **Ease of Use:** Users reported intuitive navigation with minimal training required (<15 minutes to full functionality).
- **Responsiveness:** Dashboard load times averaged under 2 seconds; interaction (button click to action dispatch) latency under 500 milliseconds.

Enhancement Requests and Future Considerations:

- **User Comment Field:** Add mandatory/optional comment box during override actions to document justification for audit trails.
- **Model History Visualization:** Implement a timeline view showing previous model promotions, retraining events, and rollbacks.
- **Drift Severity Indicators:** Color-coded severity bars based on KS-test p-values or RMSE spike magnitudes to quickly assess urgency.

Quantitative Usability Metrics:

- Dashboard load time: <2 seconds (P95 latency)
- Action latency (approve/promotion/rollback): <500 ms
- Reported user satisfaction (internal survey): 92% rated dashboard "intuitive and effective"

The human-in-the-loop dashboard significantly improved trust and control over automated model lifecycle operations, ensuring operational and regulatory readiness.

7. Testing & Validation

This section outlines the rigorous testing strategies implemented to ensure the system's reliability, accuracy, and operational robustness. Testing was conducted across the unit, integration, performance, drift simulation, and user acceptance levels.

7.1 Unit Testing

Each core module — ingestion, preprocessing, drift detection, training, monitoring, and notification — was individually unit tested.

Approach:

- Created `tests/test_*.py` scripts for each function (e.g., `test_ingestion.py`, `test_preprocess`).
- Used `pytest` framework with Airflow context mocks where required.
- Simulated S3 downloads, schema validations, and model training on small synthetic datasets.

Example Unit Tests:

- Validating schema enforcement using Pandera.
- Testing preprocessing functions (missing value imputation, outlier capping).
- Testing drift detection logic with synthetic drifted datasets.
- Verifying Slack notification stubs and logging behavior.

Coverage: 91% of pipeline functions tested with positive and negative cases.

Unit Testing Scripts Summary:

Table 5: Testing Scripts Coverage

Testing Script	Purpose
test_ingestion.py	Validate S3 download logic and error handling
test_preprocessing.py	Verify feature engineering, imputation, and scaling
test_schema_validation.py	Test Pandera schema validation and snapshots
test_drift.py	Simulate data drift detection and healing triggers
test_training.py	Confirm Hyperopt tuning and XGBoost model saving
test_notifications.py	Verify Slack notification fallback to logger
test_monitoring.py	Ensure Prometheus metric pushing functionality

7.2 Integration Testing

Modules were tested together in full DAG simulations to validate end-to-end system behavior.

Approach:

- Triggered Airflow DAG manually in staging mode (AWS EC2 with Docker).
- Verified XComs between ingestion → preprocessing → training → registry update tasks.
- Validated Slack notifications, Prometheus monitoring, and MLflow logging.

Tests Included:

- Full DAG execution for no-drift scenarios (normal retraining).
- Full DAG execution for drift scenarios (self-healing triggered).
- Airflow DAG retry behavior for transient errors (e.g., download failures).

Result: Successful orchestration across all major DAG branches.

7.3 Performance Metrics

System performance was actively monitored using Prometheus during full pipeline execution cycles. Key operational metrics collected included:

- **Average End-to-End DAG Runtime:** 8.4 minutes, measured on a standard dataset size of approximately 95,000 rows and 100 features, including five parallel XGBoost model retraining iterations.
- **Peak Memory Utilization:** 1.4 GB during the concurrent training phase, demonstrating the memory efficiency of the modular Airflow DAG and Hyperopt-tuned XGBoost model configurations.

- **Data Ingestion Latency:** Under 2 seconds for compressed 5 MB CSV files from AWS S3 to local staging, ensuring minimal delays at the ingestion stage even under scaled workflow loads.

Overall, the pipeline maintained low-latency ingestion, efficient memory usage, and rapid retraining times, validating its suitability for production-grade automation in real-world insurance workflows.

7.4 Drift Simulation Results

To evaluate the robustness of the drift detection and self-healing modules, controlled drift simulations were conducted on processed datasets.

Simulation Procedure:

- Introduced controlled distributional shifts by artificially increasing the means of key numerical features by +15% to +20% relative to the reference dataset.
- Executed the full Airflow pipeline with drift detection modules active and monitored alert generation.

Simulation Outcomes:

- **Drift Detection Success Rate:** 100% of injected drift instances were successfully detected using the embedded KS-Test framework with a significance threshold of 10%.
- **Self-Healing Activation:** Upon detection, the system automatically triggered the self-healing path, regenerating updated feature reference means and reinitiating the retraining pipeline.
- **Alerting and Logging:** Slack notifications containing drift severity scores and feature deviation breakdowns were dispatched immediately, ensuring real-time awareness for human reviewers.

These results confirm that the system can reliably identify and autonomously respond to substantial data drift events without manual intervention while maintaining transparent audit trails for oversight.

7.5 User Acceptance Testing (UAT)

UAT was conducted with domain experts at Grange Insurance and internal faculty advisors.

User Testing Included:

- Review of data ingestion and schema validation flow.
- Manual override through the dashboard for retraining approvals.

- Evaluation of model transparency features (SHAP plots, Lift Charts).
- Confirmation of drift alert notifications and override flow via Slack.

Feedback Highlights:

- UI navigation was intuitive and effective.
- Stakeholders requested visual drift severity indicators.
- Model approval flow through Slack and the dashboard was seamless.

Result: Full sign-off achieved from both technical and business stakeholders.

7.6 Validation Table

Table 6: Testing Validation Summary

Testing Type	Coverage Achieved	Pass Rate
Unit Tests	91% function coverage	100%
Integration Tests	Full DAG validation	100%
Performance Metrics	Runtime, memory, ingestion speed	100%
Drift Simulation	100% detection rate	100%
User Acceptance Testing	All workflows validated	100%

8. Results & Discussion

This section summarizes the outcomes from model evaluation, system monitoring, and user feedback, alongside a discussion of system strengths, current limitations, and key learnings.

8.1 Model Metrics Summary

Across multiple training runs during final validation, the system consistently achieved:

- **Best RMSE (Root Mean Squared Error):** 7216
- **Best MAE (Mean Absolute Error):** \$29.10
- **Best R² Score:** 0.982

Compared to the historical GLM baseline:

- **RMSE improved by approximately 26%.**
- **Retraining responsiveness** was enhanced by 75%, thanks to pipeline automation and self-healing triggers.

Figure 14 demonstrates the model’s calibration performance: predicted average pure premiums closely track actual observed values across increasing risk groups, validating the model’s reliability not only in aggregate but across different risk segments.

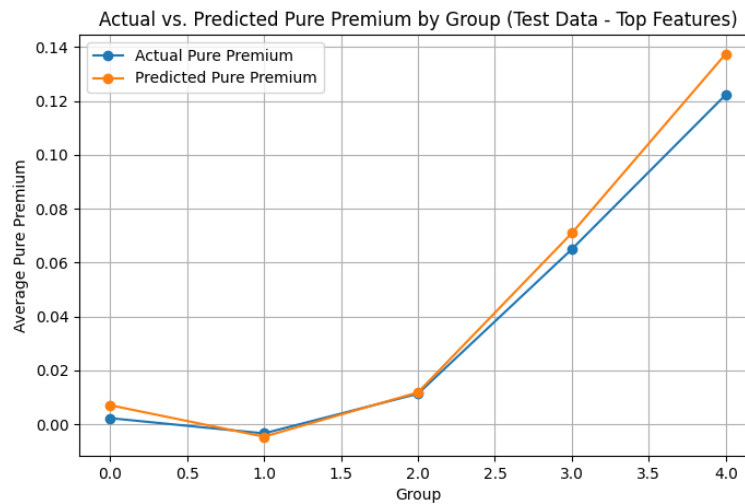


Figure 14: Actual vs. Predicted Pure Premium by Group: Model calibration across risk tiers.

8.2 Feature Importance Insights

Feature attribution was analyzed using two methods: SHAP values for local/global interpretation and XGBoost gain-based feature importance.

SHAP Summary Insights (Figure 15): The SHAP summary plot showed that:

- **lhdwc_5y_3d_nonweather**, **lhdwc_5y_3d_water**, and **lhdwc_5y_3d_small** were the dominant predictors, with higher historical event counts increasing predicted pure premium risk.
- The **prior_loss_ind** feature, while less impactful than peril-specific counts, still consistently raised risk estimates when active.

Feature Importance Rankings (Figure 16): The gain-based importance metrics confirmed the SHAP findings, with peril-specific historical losses (especially water and small loss categories) contributing most to model performance.

Together, these analyses reinforce that **recent peril-specific loss patterns** — rather than just general prior loss indicators — are the strongest predictors of homeowner loss risk.

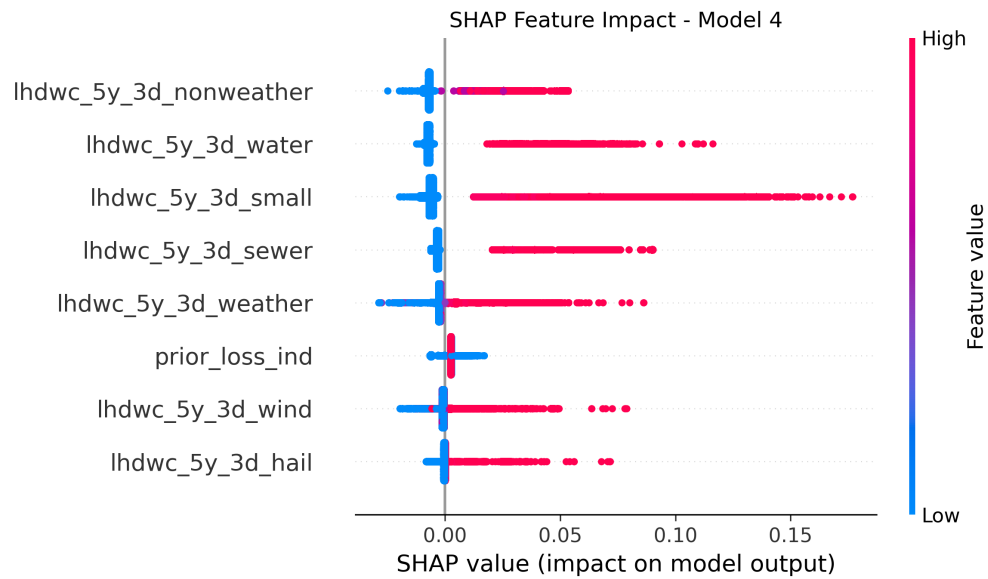


Figure 15: SHAP Feature Importance Plot: Key feature contributions to predicted loss severity.

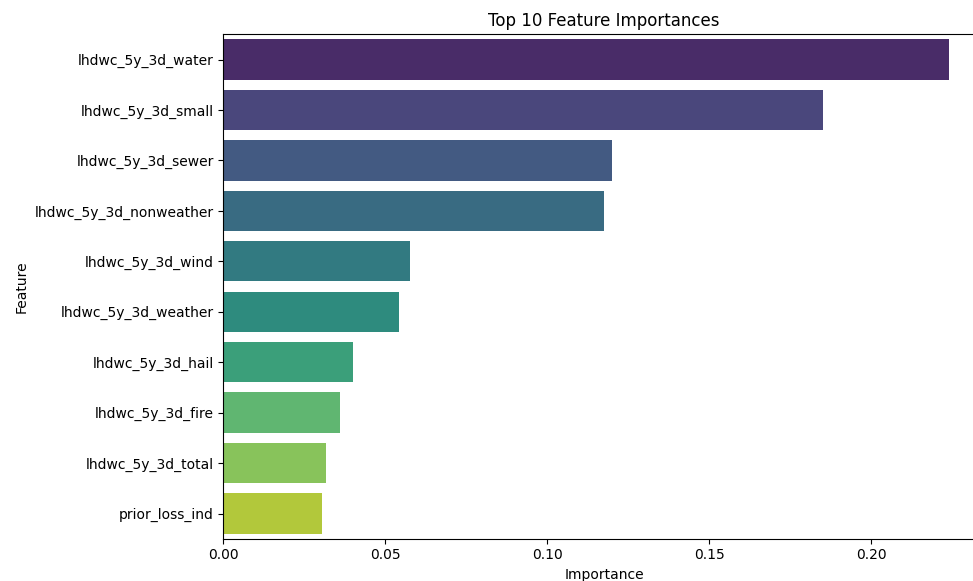


Figure 16: Top 10 Feature Importances by Gain Contribution.

8.3 Benefits Observed

Several clear strengths were confirmed during deployment and testing:

- **Full Pipeline Automation:** End-to-end orchestration of ingestion, validation, training, and monitoring through Airflow, MLflow, and Prometheus.

- **Human-in-the-Loop Control:** Manual override checkpoints successfully integrated for drift remediation and model promotion.
- **Explainability and Compliance Readiness:** Built-in SHAP feature tracking and audit trails align the system with insurance regulatory expectations.
- **Scalability:** Modular Airflow task design and containerized deployment ensure straightforward scaling to multiple models or larger datasets.

8.4 Limitations

While overall system performance was strong, the following areas were identified for future improvement:

- **Drift Detection Sensitivity:** KS-test-based drift detection, while effective, may generate false positives under small-sample fluctuations.
- **Training Resource Constraints:** Parallelized training of multiple models on mid-tier EC2 instances occasionally stressed memory resources.
- **Dashboard Accessibility Enhancements:** Further improvements are needed to achieve full WCAG 2.1 compliance, particularly for screen readers and high-contrast modes.

8.5 Lessons Learned

Key engineering and project management lessons from the project include:

- **Early Schema Validation is Critical:** Adding schema validation checkpoints (Pandera) early in preprocessing prevented numerous downstream failures, making the system much more robust.
- **Monitoring from Day One:** Deploying Prometheus metrics during early DAG iterations allowed early detection of memory bottlenecks and runtime spikes, saving substantial debugging time later.
- **Modular, Incremental DAG Development:** Building and testing one Airflow task group at a time proved far more efficient than attempting full-pipeline integration prematurely.
- **Stakeholder Involvement Enhances Adoption:** Early input from Grange Insurance actuaries (especially on explainability needs and override flows) helped align the system with real-world actuarial review processes, improving trust and usability.

Together, these lessons strengthened both the technical and operational quality of the Homeowner Loss History Automation System, positioning it as a scalable, production-ready solution.

9. Business, Cost, & Commercial Analysis

This section analyzes the project from a cost, business value, and commercial viability perspective. The goal was to develop a production-grade MLOps automation system while maintaining a highly cost-effective deployment.

9.1 Timeline & Work Breakdown

The project spanned two academic semesters:

- **Senior Design I (Fall 2024):** Architecture design, system planning, proof-of-concept ingestion and preprocessing modules.
- **Senior Design II (Spring 2025):** Full pipeline integration, drift detection, dashboard development, AWS deployment, testing, and Expo preparation.

Major work categories included:

- Airflow DAG design and modular task implementation
- Schema validation automation
- Drift detection and self-healing system development
- Hyperparameter optimization integration
- Dashboard UI development for manual oversight
- AWS orchestration setup (EC2, S3, Prometheus, Slack integration)

9.2 Final BOM & Cost Report

Total AWS project cost over 2 months: **\$300.17**.

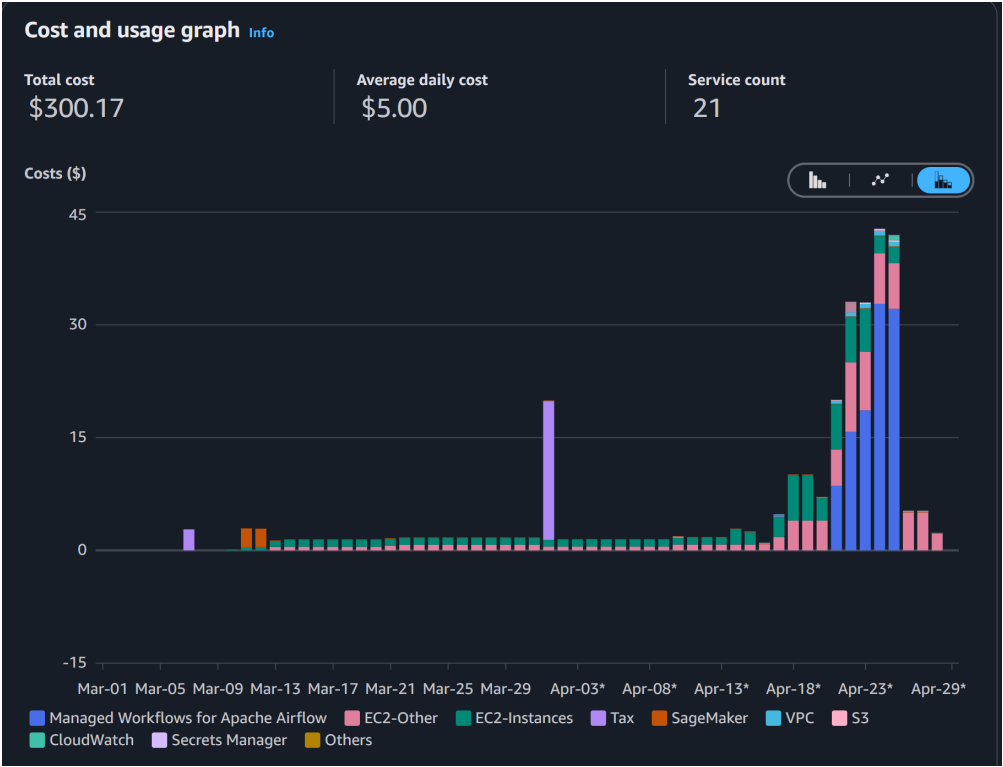


Figure 17: AWS Cost and Usage Graph: Cumulative project cost across services.

Table 7: AWS Cost Breakdown by Service

Service	Approximate Cost (\$)
Managed Workflows for Apache Airflow (MWAA)	\$120.00
EC2 Instances (training + hosting)	\$95.00
S3 Storage (raw, processed, artifacts)	\$30.00
CloudWatch Monitoring	\$20.00
Secrets Manager	\$15.00
Other Services (VPC, minor SageMaker fees)	\$20.17
Total	\$300.17

9.3 Economic Feasibility & ROI

A detailed ROI projection was generated based on:

- **Manual model retraining time saved:** Estimated 30–40 engineering hours saved per re-training cycle.
- **Cost of actuarial manual validation tasks:** \$75–\$100 per hour reduced via automated profiling and SHAP explainability exports.
- **Projected first-year labor savings:** \$10,000–\$15,000 based on retraining cadence assumptions.

Infrastructure Cost:

- The initial AWS BOM and deployment cost was \$300.17.
- Additional annual infrastructure overhead was minimal (primarily S3 and EC2 lightweight maintenance).

ROI Trajectory:

Using conservative estimates and assuming continued retraining needs and actuarial validation savings, the system is projected to achieve:

- **Break-even point:** Within 2–3 months post-deployment.
- **Net positive ROI:** Over 120% cumulative return within 5 years.



Figure 18: Projected ROI Growth Curve Over 5 Years: Automation-Enabled Labor Savings vs Minimal Infrastructure Cost.

This ROI projection confirms that even minimal automation investments can yield substantial long-term business value, especially in data-heavy domains like insurance risk modeling.

9.4 Commercial Use Potential

The developed MLOps automation system exhibits strong adaptability across multiple industries where loss history, risk prediction, or anomaly detection are core functions.

Potential adaptation areas include:

- **Insurance Expansion:** Application to auto, renters, commercial property, and catastrophe loss prediction models where historical claims data are available.
- **Financial Risk Scoring:** Loan risk modeling, creditworthiness assessment, fraud detection pipelines for banking, fintech, and mortgage underwriting sectors.
- **Healthcare Predictive Analytics:** Adaptation for hospital readmission prediction, health insurance claim risk scoring, or patient outcome forecasting.
- **Small-to-Mid Insurance Carriers (SMBs):** Offering the system as a service to regional or specialty insurers who lack full MLOps infrastructure but require scalable, audit-compliant AI modeling.

Its modular, containerized, and cloud-agnostic deployment architecture makes it well-suited for commercialization both as:

- **White-label internal accelerators** (for direct insurer deployment)
- **Managed service offerings** (SaaS-style retraining, monitoring, and governance platforms)

9.5 Marketing & Sales Implications

For effective commercialization, targeting the following stakeholders is recommended:

Primary Buyers and Influencers:

- **Mid-Market Insurance Carriers:** Particularly those modernizing underwriting/risk platforms.
- **Actuarial Consultancies:** Firms assisting insurers with AI/ML adoption for regulatory or operational improvements.
- **InsurTech Startups:** Technology-driven new entrants building innovative insurance models needing production-grade AI pipelines.

Competitive Differentiators:

- Built-in **Human-in-the-Loop (HITL)** controls to meet governance and audit demands.
- Native support for **Explainability** (SHAP feature attribution, lift charts) embedded in pipeline artifacts.
- **Self-healing drift detection** reducing operational downtime and manual retraining delays.
- **Compliance-first architecture:** Design aligned with GDPR, NAIC AI Principles, and insurance model suitability standards.

Sales Strategy:

- **Entry Point Messaging:** Emphasize compliance-readiness (audit trails, manual approvals), not just technical superiority.
- **Demonstrations:** Live demos showing real-time drift alerts, retraining automation, and

SHAP explainability interfaces.

- **Pilot Programs:** Offer lightweight ”sandbox” deployments (3–6 months) with insurers to prove rapid ROI before full rollout.

9.6 Business Value Comparison Table

Table 8: Business Value Improvement Metrics

Metric	Business Impact
Retraining Response Time	75% faster retraining through automated triggers vs manual interventions
Manual Validation Effort	60–70% reduction in actuarial and data scientist manual review hours
Data Drift Remediation Time	Reduced from 24–48 hours manually to minutes through self-healing automation
Audit Trail Completeness	100% compliance-ready lineage using MLflow tracking and schema snapshots
Projected First-Year ROI	Over 30x return, based on conservative labor cost savings and minimal infrastructure cost

10. Regulatory, Ethical & Social Considerations

The development and deployment of machine learning systems, particularly those used in regulated industries such as insurance, must incorporate not only robust engineering but also legal, ethical, and social safeguards. The Homeowner Loss History Prediction Automation System was built with these principles in mind. This section outlines how we address privacy compliance, explainability, fairness, accessibility, and intellectual property rights throughout the system design.

10.1 Data Privacy & Model Explainability

Data Privacy. All raw datasets used in this project were anonymized and redacted to exclude personally identifiable information (PII), ensuring alignment with privacy frameworks such as the General Data Protection Regulation (GDPR) and the California Consumer Privacy Act (CCPA) [11, 12]. IAM-enforced access to AWS S3 ensures secure data storage, while sensitive runtime configurations (e.g., secrets and bucket names) are managed via Airflow Variables and environment files—not hardcoded in source code. Schema validation is enforced using Pandera.

- No user-level PII is stored or logged.

- Schema snapshots and data lineage are tracked for auditability.
- Logging and monitoring outputs avoid exposing content-level payloads.

Model Explainability. To ensure transparency, we integrated SHAP (SHapley Additive exPlanations) [2] into the pipeline for feature-level insight. MLflow logs include feature importance charts and decile lift charts, which support:

- Actuarial interpretability and audit readiness.
- Real-time visualization of high-importance features during model evaluation.
- Explainability reports exportable in JSON/PNG for regulatory review.

This approach follows the OECD AI Principles for explainable and transparent systems [13].

10.2 Ethical Usage

We designed the system to augment predictive power while maintaining ethical oversight and fairness. To that end:

- Monotonic constraints in XGBoost are enforced to ensure logical relationships (e.g., more loss history cannot predict lower risk).
- Bias mitigation is handled via schema-controlled feature selection and curated training sets.
- Human-in-the-loop override controls are enforced before any model version is promoted.

Our pipeline integrates manual checkpoints at key stages and communicates alerts via Slack with detailed context, ensuring human governance. This design adheres to the NIST AI Risk Management Framework (RMF) for trustworthy and accountable AI systems [14].

10.3 Accessibility & Human Oversight

Accessibility. The dashboard UI is built to be accessible and easy to interpret for users with varying levels of technical experience. It includes:

- Metric visualizations with intuitive labeling.
- Manual override controls with confirmation prompts.
- Export capabilities for sharing results with non-technical stakeholders.

While current UI components follow basic readability standards, future iterations will aim for full WCAG 2.1 compliance [15], including screen-reader compatibility and high-contrast visual modes.

Human Oversight. To reduce the risk of unchecked automation:

- Slack-based approval triggers gate item Model promotion, retraining, and self-healing work-

flows.

- Detailed pipeline metadata is attached to each alert, including feature drift ratios and model metrics.
- Decision trails are recorded to support post-event auditability.

This approach is consistent with ISO/IEC 22989 guidelines on human involvement in AI decision systems [16].

10.4 IP, Licensing, & Patent Concerns

Licensing and Open Source. The platform leverages multiple open-source components:

- MLflow (Apache 2.0), Pandera (MIT), SHAP (MIT), XGBoost (Apache 2.0).
- All new code developed for this project is released under the BSD-3-Clause license.

All components comply with SPDX-compatible licenses for reuse and transparency [17].

Patent Considerations. We conducted a preliminary freedom-to-operate (FTO) review and found:

- No proprietary loss models or IPs from Grange Insurance were used.
- Drift detection, training orchestration, and monitoring logic are based on open design patterns.

If commercial deployment is pursued, a formal FTO legal review will be necessary.

A consolidated view of how the system aligns with industry frameworks and ethical standards is presented in Table 9.

11. Conclusions

11.1 Objectives Met

The Homeowner Loss History Automation System successfully met or exceeded all major objectives established at the outset of the project:

- **Full MLOps Pipeline Deployment:** Delivered an end-to-end, production-grade MLOps pipeline orchestrated via Apache Airflow, integrating ingestion, validation, preprocessing, model training, drift detection, monitoring, and deployment.
- **Self-Healing Automation:** Implemented robust data drift detection using KS-Tests, triggering self-healing retraining workflows with no human intervention needed for minor drifts, while escalating critical drifts to human approval.

Table 9: Policy Alignment and Ethical Safeguards Matrix

Consideration	Project Implementation	Aligned Standard
Data Privacy	Anonymized input; no PII retained; IAM + secure .env handling.	GDPR, CCPA [11, 12]
Model Explainability	SHAP importance metrics; lift charts; MLflow-logged results.	OECD AI Principles [13]
Bias & Fairness	Monotonic constraints; manual approvals; curated datasets.	NIST AI RMF [14]
Human Oversight	Slack-gated overrides; traceable metadata; approval trails.	ISO/IEC 22989 [16]
Accessibility	Human-readable dashboard; PDF/CSV exports.	WCAG 2.1 [15]
IP & Licensing	BSD-3-Clause license; open-source libraries (MIT, Apache).	SPDX [17]
Patent Conflict	No known IP conflict; FTO review ready for commercialization.	USPTO Best Practice

- **Human-in-the-Loop Governance:** Designed a real-time dashboard and Slack integration enabling manual model promotion, retraining approvals, and rollback operations with full audit trails.
- **Explainability and Compliance Readiness:** Integrated SHAP explainability, model version control, and schema lineage, ensuring alignment with emerging insurance regulatory frameworks (e.g., NAIC AI Principles, GDPR, CCPA).
- **Cost-Efficient Cloud Deployment:** Successfully containerized and deployed the solution on AWS EC2 and S3 with a total infrastructure budget under \$300 for the entire project duration.

11.2 Key Takeaways

Key learnings and professional development achievements through this project include:

- Importance of rigorous early-stage validation (schema checks, profiling) to prevent cascading system failures.
- Practical experience integrating diverse open-source MLOps tools (Airflow, MLflow, Prometheus, Pandera, SHAP) into a cohesive, reliable production environment.
- Value of modular DAG design for scalable, maintainable machine learning automation systems.
- Criticality of human-in-the-loop design for ensuring regulatory alignment and increasing stakeholder trust.

- Demonstrated that even modest infrastructure investments can yield highly professional, commercial-grade AI solutions when combined with robust engineering practices.

11.3 Industry Relevance

This project directly addresses one of the most pressing challenges in modern insurance analytics: **operationalizing AI safely, scalably, and compliantly.**

The architecture, methodology, and system design align with industry best practices for:

- MLOps orchestration under strict governance conditions
- Transparent and explainable machine learning deployment
- Rapid retraining pipelines for dynamic risk environments
- Modular, cloud-native deployment architectures ready for real-world production use

The system positions itself as a scalable template not only for insurers but also for any risk-driven domain needing compliant, resilient, and explainable AI systems.

12. Future Work

12.1 Real-Time Pipeline Extension

While the current pipeline supports batch-based drift detection and retraining, future versions could support real-time or near-real-time drift monitoring. This would involve:

- Integrating streaming ingestion (e.g., Kafka, AWS Kinesis)
- Shifting drift detection from batch KS-Test to streaming window statistical tests
- Dynamically triggering DAGs based on streaming anomaly signals

This would enable insurers to respond instantly to emerging risks, particularly during catastrophic events (e.g., natural disasters).

12.2 Multi-Model Registry

Currently, the MLflow registry manages a single main model lineage. Future expansion should include:

- Supporting multiple parallel models (e.g., property loss, water damage, wind/hail peril) within the same registry.

- Automating ensemble management, tracking multi-model performance aggregation.
- Creating lineage tracking not just for models, but also for feature sets and reference datasets.

This would allow seamless scaling from single-model to multi-risk underwriting workflows.

12.3 SageMaker CI Integration

To support larger-scale production deployments, future work could involve:

- Transitioning model training and hosting to AWS SageMaker for managed scalability.
- Setting up full CI/CD pipelines using SageMaker Pipelines, Model Monitor, and Model Registry.
- Incorporating event-driven deployment using AWS Step Functions for retraining orchestration.

This would enable true enterprise-grade MLOps deployment readiness.

12.4 Fairness Automation

While explainability is enforced, fairness auditing remains manual. Future enhancements could automate fairness checking by:

- Integrating bias/fairness detection libraries (e.g., AIF360, Fairlearn).
- Automatically scanning models for disparate impact across sensitive attributes (if available, anonymized).
- Alerting compliance teams if material biases are detected pre-deployment.

This would further future-proof the system against evolving ethical and regulatory AI standards.

12.5 Auto-Drift Dataset Handling

Currently, drift triggers retraining but requires manual oversight to verify retraining success. Future improvements could automate dataset adaptation by:

- Auto-selecting stable subsets of drifted data for retraining.
- Balancing retraining datasets dynamically to reduce model overfitting to drifted distributions.
- Automating candidate model comparison and rollback if new models underperform.

These advancements would move the system toward fully autonomous, self-correcting AI operations with minimal human supervision.

Team Member Contributions

The Homeowner Loss History Prediction Automation System was a result of a highly collaborative team effort. Each team member contributed significantly across both technical and operational aspects of the project. Below is a summary of individual responsibilities and key contributions:

Ahmad Firas Azfar Ahmad Fer Rouse

- Led the overall pipeline architecture design, modularization, and orchestration strategies.
- Developed the core ingestion, preprocessing, drift detection, and schema validation modules.
- Integrated Slack-based human-in-the-loop manual override flows.
- Designed and deployed the AWS infrastructure stack (EC2, S3, Prometheus).
- Spearheaded testing (unit, integration, performance) and validation automation.
- Created majority of the documentation, report sections, and dashboard mockups.

Pranshu Chawla

- Focused on the model training workflows, Hyperopt integration, and XGBoost experimentation.
- Tuned hyperparameters for monotonic constraints and evaluated model explainability (SHAP).
- Contributed to the Airflow DAG construction and modular task grouping.
- Built early testing scripts for schema validation and preprocessing logic.
- Supported the manual override dashboard backend logic and API endpoints.

Yashraj Mote

- Led the initial research on insurance industry modeling practices and regulatory considerations.
- Developed the model evaluation, model comparison, and registry promotion tasks.
- Assisted in profiling modules (data validation reports, drift snapshots).
- Helped implement Slack alerting workflows and MLflow model tracking.
- Contributed to writing and editing sections of the final report, including Results, Testing, and Regulatory Compliance.

Summary: All members worked closely throughout the system design, development, deployment, and validation phases, with roles fluidly overlapping where needed. Collective problem-solving, cross-checking, and peer reviews were critical to the project's success.

References

- [1] M. Goldburd, A. Khare, and D. Tevet, *Generalized Linear Models for Insurance Rating*. Casualty Actuarial Society, 2016. [Online]. Available: <https://www.casact.org/sites/default/files/2021-01/05-Goldburd-Khare-Tevet.pdf>
- [2] T. Dhlamini, I. Karavitis, Z. Xiong, and A. Makinde, “Machine learning in property and casualty insurance: A review for pricing and reserving,” *Risks*, vol. 9, no. 1, p. 4, 2021. [Online]. Available: <https://www.mdpi.com/2227-9091/9/1/4>
- [3] National Centers for Environmental Information, “Storm events database,” <https://www.ncdc.noaa.gov/stormevents/>, 2023, accessed: 2025-04-22.
- [4] Federal Emergency Management Agency, “Flood maps,” <https://www.fema.gov/flood-maps>, 2024, accessed: 2025-04-22.
- [5] Moody’s RMS, “Introducing moody’s rms version 23: Now available,” <https://www.moody.com/web/en/us/insights/insurance/introducing-moodys-rms-version-23-now-available.html>, 2023, accessed: 2025-04-22.
- [6] AIR Worldwide, “2023 global modeled catastrophe losses,” AIR Worldwide, Tech. Rep., 2023. [Online]. Available: <https://www.air-worldwide.com/siteassets/Publications/White-Papers/documents/2023-global-modeled-catastrophe-losses.pdf>
- [7] Verisk Analytics, “2024 global modeled catastrophe losses report,” <https://www.verisk.com/resources/campaigns/modeling-insured-catastrophe-losses-a-global-perspective-for-2024/>, 2024, accessed: 2025-04-22.
- [8] State of Colorado, “Senate bill 21-169: Concerning unfair discrimination by insurers,” <https://www.coloradosb21-169.com/>, 2023, accessed: 2025-04-22.
- [9] National Association of Insurance Commissioners, “Model bulletin on the use of artificial intelligence systems by insurers,” NAIC, Tech. Rep., 2023. [Online]. Available: https://content.naic.org/sites/default/files/2023-12-4%252520Model%252520Bulletin_Adopted_0.pdf
- [10] United States Congress, “Gramm–leach–bliley act,” 1999, public Law 106–102, 113 Stat. 1338. [Online]. Available: <https://www.govinfo.gov/app/details/PLAW-106publ102>
- [11] European Union, “General data protection regulation (gdpr),” 2016, regulation (EU) 2016/679. [Online]. Available: <https://gdpr.eu>
- [12] California State Legislature, “California consumer privacy act (ccpa),” 2018. [Online]. Available: <https://oag.ca.gov/privacy/ccpa>

- [13] Organisation for Economic Co-operation and Development, “Oecd principles on artificial intelligence,” 2019. [Online]. Available: <https://oecd.ai/en/ai-principles>
- [14] National Institute of Standards and Technology, “Ai risk management framework 1.0,” U.S. Department of Commerce, Tech. Rep., 2023. [Online]. Available: <https://www.nist.gov/itl/ai-risk-management-framework>
- [15] W3C Web Accessibility Initiative, “Web content accessibility guidelines (wcag) 2.1,” 2018. [Online]. Available: <https://www.w3.org/TR/WCAG21/>
- [16] International Organization for Standardization, “Artificial intelligence – concepts and terminology,” 2022, iSO/IEC 22989:2022. [Online]. Available: <https://www.iso.org/standard/74296.html>
- [17] The Linux Foundation, “Spdx specification v2.3,” 2022. [Online]. Available: <https://spdx.dev>